CallVerse, Explained Simply

Oct 21, 2026 · @woujoud

How to read this guide

This is the simple version of the 66-page CallVerse Technical Guide: the same project, in everyday words, one idea at a time.

- **Go slowly, from top to bottom.** Each part only uses ideas that were explained before it.
- **Read the first sentence of each part twice.** It is the main point. If you remember only that sentence, you remember the most important thing.
- **Hard words are explained the first time they appear.** They are all collected again in the Word list at the end.
- **The numbers are the real CallVerse results.** When a number is only there to help you understand, the text says "small example".
- **The original guide still has the screenshots.** Each part here ends with the matching part of the original, so you can look at the pictures there.

The order follows the path of the data. First the story and the goals (parts 1 to 3). Then how each piece works (parts 4 to 11). Then the checks, the screens and two full journeys (parts 12 to 14), and the commands (part 15).

If something is still unclear, add a comment on that sentence and ask for a simpler explanation.

1. CallVerse in one minute

CallVerse is a computer copy of an online shop's customer-service centre, used to test decisions before trying them on real customers.

The problem. Picture the help centre of a big online shop. Customers write when an order is late, broken or wrong, or when they want their money back. The centre can fail in two ways:

- **Too few advisors** during sales (like Black Friday) or delivery problems. Customers wait, give up and complain.
- **Too many advisors** on quiet days. The company pays people who have nothing to do.

Managers must decide three things: how many people to schedule, which advisor takes which customer, and what an advisor is allowed to promise. Usually they decide by feeling, and they test the idea directly on real customers. If the idea is bad, real customers suffer.

The idea: a digital twin. A *digital twin* is a copy of a real system that lives inside the computer. Pilots train in a flight simulator before they fly a real plane. In the same way, CallVerse lets a manager try an idea on pretend customers first, look at the result, and only then decide.

What CallVerse does, in five steps:

1. It guesses how many customers will write each day. This is the *forecast*.
2. It plans who works when. This is the *schedule*.
3. It plays the whole week again, minute by minute, with pretend customers and pretend advisors. This is the *simulation*.
4. It lets AI helpers, called *agents*, answer the customers and grade the answers.
5. It compares every new idea with a simple, normal way of working, and uses statistics to say if the difference is real or just luck.

Original guide: part 1.1.

2. The four goals, and which ideas won

CallVerse has four business goals: two new ideas won clearly, one goal kept its safer backup, and one kept a mix of both.

How each goal is organized. The project follows a fixed chain: business goal (*BO*) → business number to watch (*KPI*) → data-science goal (*DSO*) → measure (*metric*). Then each goal gets four things:

- a **main idea**: the new, smart method;
- a **baseline**: the simple, normal method that the main idea must beat;
- a **backup**: a safer method, used if the main idea fails;
- a **switch rule**: a test, fixed before the experiment, that decides which method is kept.

The switch rule is the important part. The choice is never made by opinion; it is made by a number.

Goal	The new idea, and the simple method it must beat	What happened	Kept
Be ready for busy days (BO4 / DSO4)	A smart forecast and the digital twin, against "same day last week"	Forecast error 19.9% smaller. The twin passes every realism check.	Main idea

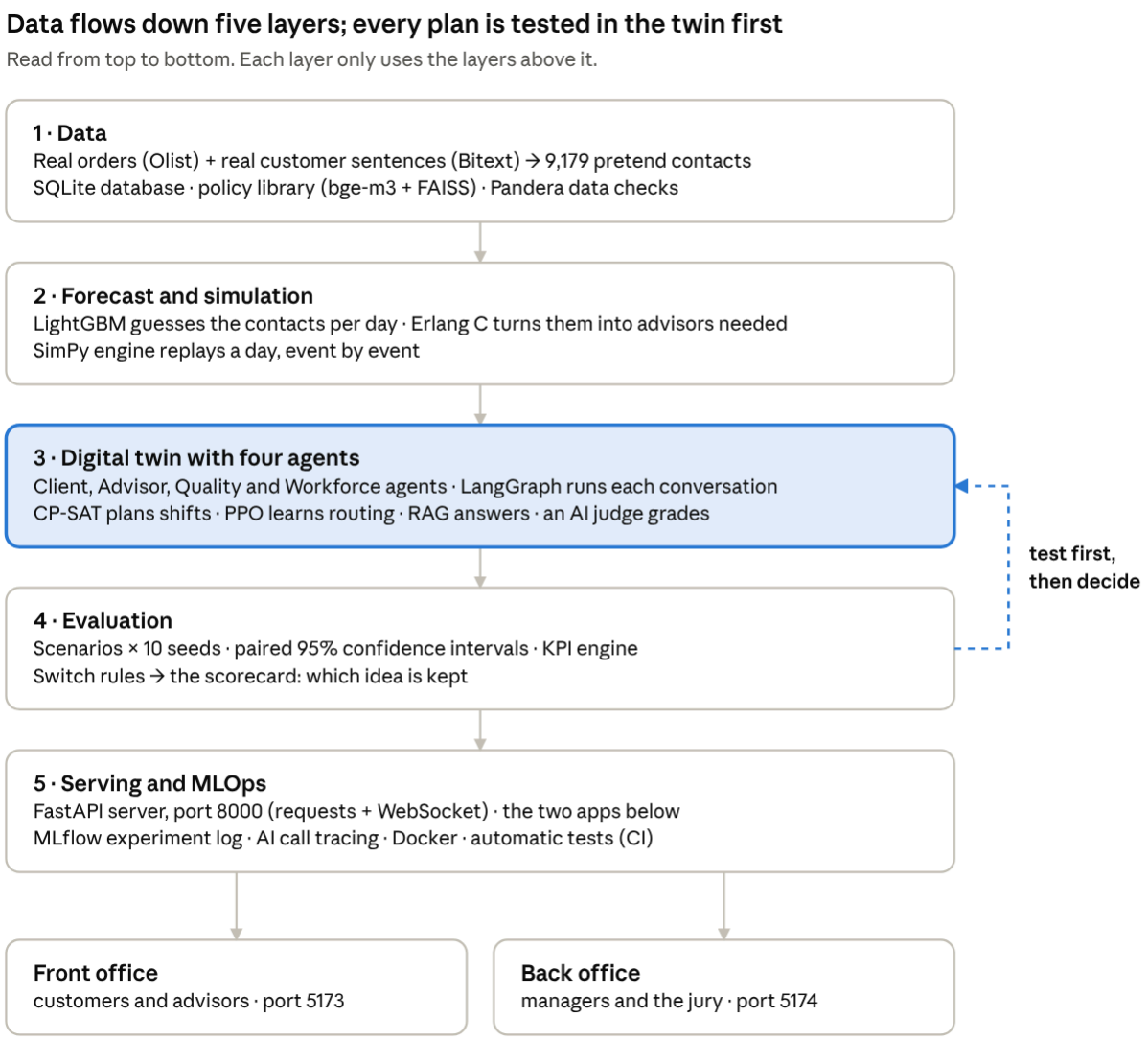
Goal	The new idea, and the simple method it must beat	What happened	Kept
Lower cost, same service (BO3 / DSO3)	An automatic shift planner, against a fixed schedule sized for the busiest hour	Labour cost 40.1% lower, and 92.4% of chats answered within 20 seconds (target: 80%)	Main idea
Shorter waiting (BO1 / DSO1)	An AI that learns to choose the next customer, against "first come, first served"	Better on hard single days, but not on whole busy weeks, so the rule is not met	Backup
Correct answers, happy customers (BO2 / DSO2)	An AI advisor that reads the company policies, against keyword search and fixed answers	The search is much better, but the small AI alone invents order numbers and promises. The AI judge is not trusted yet.	A mix: the rules decide, the AI writes

Is "backup kept" a failure? No. The rule was written before the test. The new idea did not pass it, so the safer method is used and the new idea stays as research. Saying this honestly is part of good science.

Original guide: parts 1.2 and 1.3.

3. The big picture: five layers and two apps

CallVerse is built in five layers, one after the other: data goes in at the first layer, and the last layer serves the two apps that people use.



CallVerse architecture · 5 layers, 2 apps

The dashed arrow is the main idea of CallVerse: every plan goes back to the twin and is tested there before anyone decides.

Two rules keep the layers clean.

- Each layer is one folder of Python code in `src/callverse/`. A layer only uses the layers that come before it, never the ones after it. For example, the data layer may not use the agents. An automatic check (five *import-linter* rules) tests this on every change.
- Every assumption (how often customers write, opening hours, costs, model names) is written in a settings file in `configs/`, not hidden in the code. These files use the YAML format.

Two apps on one server. An *API* is the list of requests a server accepts, like a menu in a restaurant: the apps order, the server answers. Both apps use the same API.

App	Address on your computer	Who uses it	What it is for
Front office	localhost:5173	Customers and advisors	The "shop side": help centre with search, a demo customer account, a live chat with the AI advisor, and an advisor desk with an AI helper
Back office	localhost:5174	Managers, data scientists, the jury	The "control room": forecast, digital twin, schedules, routing, quality, scores, models
API (the server)	localhost:8000	Both apps	Answers the apps' requests; its documentation is at localhost:8000/docs

Original guide: parts 1.4 and 1.5.

4. The data: where the pretend customers come from

CallVerse starts from real orders and real customer sentences, and creates 9,179 customer contacts from them with clear, written rules.

Why create the contacts? No public dataset has real customer-service chats linked to real orders. So CallVerse mixes two real public datasets:

- **Olist**: 99,441 real orders from a Brazilian online marketplace (2016 to 2018). For each order we know when it was bought, paid, given to the carrier and delivered, the promised delivery date, the payment, the items and the customer's review.
- **Bitext**: real customer-service sentences. Each one is labelled with its *intent*, which means what the customer wants. CallVerse keeps 17 intents about orders and groups them into four *problem families*:

Problem family	What the customer wants
wismo ("where is my order?")	To know where the order is
cancel_refund	To cancel the order or get the money back
post_delivery	Help with a damaged, wrong or missing item, or a return
order_admin	An invoice, an address change, or help with a payment

How one contact is created, step by step:

1. The program looks at each real order and finds signs of trouble: the delivery is late, the seller shipped late, the order was cancelled, the payment looks strange, the review score is low, the transport took a long time.
2. Each sign of trouble has a chance that the customer writes, and a moment when they write (for example, 2 days after the shipping deadline). These numbers are in the settings file `configs/data.yaml`.
3. A random draw decides if this customer really writes. The draw uses a fixed *seed* (the starting number of the random generator), so you get the same result every time you run it.
4. The contact gets an intent that matches the trouble, and the real sentence of a Bitext customer with that intent.
5. Sales periods (Black Friday, Christmas) and public holidays change how many people write.

The result. 9,179 contacts from January 2017 to August 2018, each linked to a real order. From them, CallVerse also creates 9,179 tickets and 60,837 chat messages:

- a *rules engine* (a program that contains the company's rules) decides the correct outcome, for example share the tracking, approve a refund, open an investigation with the carrier, or hand the case to logistics;
- approved answer templates write the advisor's messages;
- 30% of customers answer a satisfaction survey (*CSAT*, a score from 1 to 5).

Every created row is marked `is_synthetic = True`, so nobody can mix it up with real data.

Where the data is kept:

- **The database** (`data/callverse.db`, a SQLite file) holds the orders, tickets and chats. The advisor's tools read it "as of" the moment the customer writes. So if an order was "shipped" on that day, the advisor never sees "delivered", even if it arrived later.
- **The knowledge base** holds 10 policy documents written by hand: refunds, returns, delays, invoices, payments, cancellations, tracking, FAQ, damaged items, and the advisor playbook. Each section of a document becomes one *chunk*: 58 chunks in total. Part 10 explains how they are searched.
- **Data checks** (*Pandera contracts*): after every run, each table is checked against rules (right types, values in range, unique ids, allowed values) before the forecast and the twin use it. It works like a quality check at a factory gate.
- **The data notebooks** (03 to 08) are the single source of truth. The command `callverse data` runs them again automatically.

Original guide: parts 2.1 and 4.2.

5. The forecast: guessing how many customers will write

CallVerse predicts how many contacts each problem family will bring on each of the next 7 days, and its model makes the smallest error of all the methods tested.

Why forecast? To plan next week's advisors, you first need to know how many customers will come. The forecast looks 1 to 7 days ahead, because the schedule is made one week at a time.

The clues the model uses (called *features*):

- how many contacts there were 7, 14, 21 and 28 days before;
- orders, deliveries, late deliveries and the pile of overdue orders, with the same delays;
- the day of the week, promotion days and public holidays.

No peeking at the future. Every clue is at least 7 days old. So a forecast made on Sunday for next Saturday only uses what was really known on Sunday. If a model sees the future by mistake (this is called *leakage*), it looks great in tests and fails in real life. A unit test checks that the clue `lag_7` really equals the value from 7 days earlier.

The model: LightGBM. LightGBM builds many small decision trees, one after the other. Each new tree corrects the mistakes of the trees before it. One single model learns the four problem families together (the family is one of its clues). It uses a *Poisson* setting, which tells it that it predicts counts: 0, 1, 2, 3 contacts and so on.

The simple methods it is compared with (called *baselines*):

- "same day last week" (called *seasonal naive*): the official method to beat;
- the average of the same weekday over the last 4 weeks;

- SARIMA and Prophet: two classic forecasting methods.

How the test works: the rolling-origin backtest. Imagine you go back in time:

1. Stand on a Sunday in the past. Train each model only on the data before that Sunday.
2. Forecast the next 7 days.
3. Compare with what really happened.
4. Move to the next Sunday and do it again.

CallVerse does this every Sunday from October 2017 to August 2018: 46 times. Then it adds up the errors of all 46 tests.

How the error is measured: wMAPE. It is the total of the errors divided by the total of the real contacts:

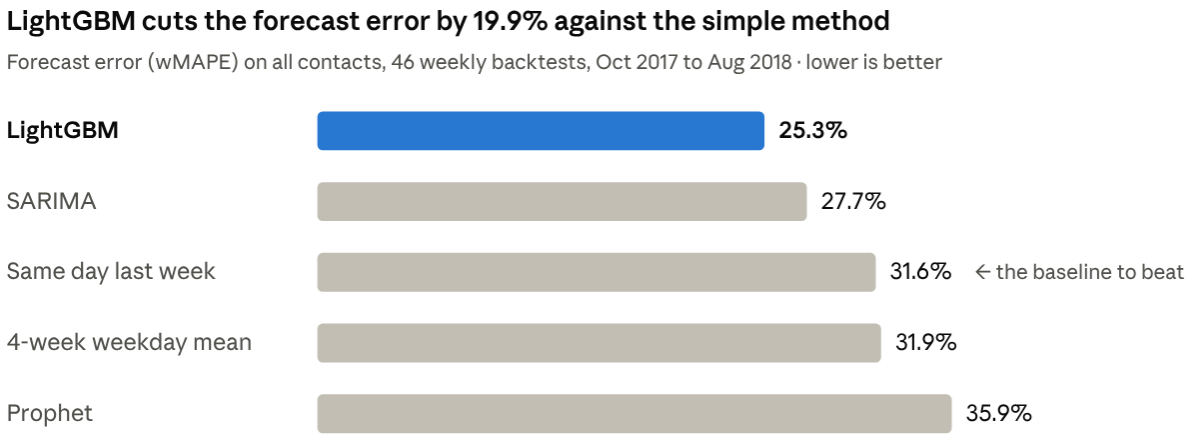
wMAPE = Σ |real − forecast| ÷ Σ real

Here Σ (the Greek letter sigma) means "add them all up", and |real − forecast| is the size of each error, without its sign (too high and too low count the same).

Small example: over 3 days the real numbers are 100, 50 and 50, and the forecast said 90, 60 and 50. The errors are 10, 10 and 0, so 20 in total. The real total is 200, so wMAPE = 20 ÷ 200 = 10%.

Why not the usual MAPE? MAPE divides each day's error by that day's value. On a day with only 1 contact, an error of 3 becomes 300%, and the average explodes. wMAPE weights each day by its volume, so quiet days cannot break the score.

CallVerse also reports MAE (the average error, in contacts), RMSE (like MAE, but big errors count more) and bias (does the model guess too high or too low on average?). But wMAPE is the one that decides.



CallVerse Technical Guide, part 2.2 · wMAPE on the total, 5 methods

LightGBM also beats "same day last week" at every distance, from 1 day to 7 days ahead. On the small families (a few contacts a day) the numbers are mostly chance, so the decision is made on the total. Every run is recorded in MLflow (part 11).

Original guide: parts 2.2 and 4.3.

6. How many advisors do we need? (Erlang C)

Erlang C is a classic formula that turns "how many customers arrive per hour" into "how many advisors we need", so that 80% of chats are answered within 20 seconds.

A call centre is a queue. Customers arrive, wait in line, and are served by advisors. Two numbers describe each hour:

- **λ** (the Greek letter lambda): how many contacts arrive per hour;
- **AHT** (average handle time): how long one conversation takes, on average.

Multiply them and you get the **load**: A = λ × AHT. It is measured in *Erlangs*, and it means "how many advisors would be busy all the time".

Small example (not CallVerse's real numbers). 60 chats arrive per hour, and each one takes 6 minutes, which is 0.1 hour. The load is A = 60 × 0.1 = 6. So 6 advisors would be busy non-stop.

But 6 advisors are not enough. Customers do not arrive evenly: sometimes three arrive in the same minute, sometimes nobody comes for five minutes. With exactly 6 advisors, the line would grow without end. The Erlang C formula gives the chance that a customer must wait, for any number of advisors:

Advisors	Chance of waiting	Answered within 20 seconds
7	61%	42%
8	36%	68%
9	20%	83%

So, in this example, the centre needs 9 advisors, not 6. The 3 extra people absorb the random bursts.

The formula. You never compute it by hand; the code does it. With c advisors and a load A:

- **Chance of waiting:** P(wait) = X ÷ (1 + A + A^2/2! + … + A^(c−1)/(c−1)! + X), where X = A^c/c! × c/(c − A)
- **Service level:** 1 − P(wait) × e^(−(c − A) × T/AHT)
- **ASA:** P(wait) × AHT ÷ (c − A)

Here A^c means A multiplied by itself c times, c! ("c factorial") means 1 × 2 × … × c, and e is a fixed number, about 2.718.

T is the target time (20 seconds). *ASA* (average speed of answer) is the average waiting time.

CallVerse uses Erlang C in three ways:

1. **To plan.** For each hour, the number of advisors needed is the smallest number that reaches the "80/20" target (80% of chats answered within 20 seconds), while keeping advisors busy at most 90% of their time. That share of busy time is called *occupancy*.
2. **To inform.** The Workforce agent tells each customer the expected wait.
3. **To check the simulator.** On a calm, steady hour, the digital twin must give the same answer as the formula. The difference is below 4%.

Little's law, in one line: the average number of people waiting = the arrival rate × the average wait (L = λ × W). This is why "make the total waiting time small" is the same goal as "make the average wait small". The routing AI uses this idea for its score (part 8).

Original guide: part 2.3.

7. The digital twin: a pretend call centre inside the computer

The digital twin replays a whole week of customers and advisors inside the computer, so a plan can be tested on pretend customers before real ones.

How time moves. The twin is a *discrete-event simulation*, written with the SimPy library. Time does not tick second by second. It jumps from one event to the next: a customer arrives, an advisor becomes free, a customer gives up, a shift starts. It is like skipping straight to the next important moment in a film.

What is inside the twin:

Part	How it works
Arrivals	Each day's volume comes from history or from the scenario, spread over the hours of the day. Arrivals are random but realistic: sometimes several at once, sometimes none (a *Poisson process*).
Live or overnight	Chats that arrive between 8:00 and 22:00 are live. Messages that arrive at night wait in an overnight pile (the *backlog*), handled first in the morning.

Part	How it works
Customers	Three personalities (*personas*): impatient 25% (half the normal patience), normal 55%, patient 20% (twice the normal patience). When patience runs out, the customer leaves. Customers who write again are more often impatient.
Conversation length	Random, with a few very long conversations, different for each problem family (a *lognormal* curve).
Advisors	Each advisor has one main skill: faster on their specialty (time × 0.75) and slower on weak topics (time × 1.3). A duty table says who works each hour.
Overtime	Planned extra hours, plus an emergency rule: if more than 8 customers wait, 2 on-call advisors join.
Routing	When an advisor is free, a *router* picks the next customer (part 8).
Costs	€12 per hour of normal work, overtime costs 1.5 times more, and each lost customer costs €8.

What the twin measures (the *KPIs*):

- **Service level**: share of chats answered within 20 seconds.
- **ASA**: average wait of the customers who got an answer.
- **Mean wait** and **p90 wait** (90% of customers waited less than this).
- **Abandon rate**: share of customers who left before an answer.
- **Occupancy**: share of working time the advisors spend in conversations.
- **Understaffed hours**: hours with fewer advisors than Erlang C asks for.
- **Money**: overtime hours, labour cost, cost per contact, total cost.

Scenarios: "what if...?" A *scenario* is a real past week with something changed: a promotion spike, delivery delays, a staff shortage (some advisors absent), or Black Friday 2017. Scenarios are written in `configs/scenarios.yaml`.

Prepared or unprepared: the key test of goal BO4.

- An **unprepared** plan is built from the normal forecast, as if nobody expected the event.
- A **prepared** plan is built after testing the scenario in the twin first.

Then both plans face exactly the same customers, and the difference shows the value of testing first. Testing first cuts the understaffed hours by 60% to 84%. Preparing for a peak saves €730 (net) over three peak weeks.

One promo day, seen on the live floor (an animated page of the back office, with the "first come, first served" router):

Promo day	Unprepared	Prepared
Advisors at opening (8:00)	8	11
Overnight messages still waiting at 08:32	78	47
Service level for the whole day	83.6%	93.1%
Customers who hung up	64	28

At 08:32 in the unprepared plan, all 8 advisors are busy with overnight messages. The live customers wait behind them: 14 have already hung up, and nobody has been answered yet.

A fair comparison: same customers, 10 runs. The twin is random, so one run could be lucky. So CallVerse does this:

1. It simulates each option 10 times, with 10 different seeds.
2. With the same seed, every option meets exactly the same customers: same arrival times, same patience, same conversation lengths. This is called *common random numbers*.
3. It computes the difference between two options seed by seed: 10 differences.
4. From these 10 differences, it computes a *paired 95% confidence interval*: a range where the true difference very likely lies.
5. If that range does not contain 0, the difference is called *significant*: it is real, not luck.

Think of comparing the fuel use of two cars. If you drive both on the same road, on the same day, the road cannot explain the difference: only the car can. Pairing does the same with customers, so even small effects become visible.

Is the twin realistic? Three checks compare it with the real history:

- **Daily volumes**: the *Kolmogorov–Smirnov test* checks if two lists of numbers follow the same pattern. Here p = 0.998, which means no visible difference.
- **Shape of the day** (share of contacts per hour): the distance between simulated and real is only 0.015 (0 would mean identical).
- **Erlang C**: on steady hours, the twin's chance of waiting and its ASA are within 4% of the formula.

Original guide: parts 2.4, 4.4 and 4.5.

8. The Workforce agent: who works when, and who answers whom

The Workforce agent has two jobs: it builds the weekly schedule, and it chooses which waiting customer each free advisor takes next.

Job 1: the weekly schedule (goal DSO3)

Input: the number of advisors needed each hour (Erlang C applied to the forecast).
Output: who works when.

The planner uses *CP-SAT*, a solver from Google OR-Tools. A solver searches through a huge number of possible combinations very fast, and it can prove that its answer is the best one. It works in two stages:

1. **Coverage.** Choose how many people start each of 27 shift types (4, 6 or 8 hours long) each day, plus some overtime (at most 4 per hour and 60 per week). The goal is the smallest total: normal pay + 1.5 × overtime pay + a big penalty (weight 200) for each missing advisor-hour.
2. **Roster.** Give the chosen shifts to named advisors, following hard labour rules: at most 5 days and 40 hours a week, one shift per day, and at least 11 hours of rest between two shifts.

The solver finds the best plan, and proves it, in less than one second.

A detail that matters. The solver runs with one worker (`num_workers = 1`). With 8 workers in parallel, it sometimes returned different plans from run to run. They were all equally cheap, but they changed the simulation results. Getting the same result every time matters more than speed here.

What it is compared with:

- a fixed schedule sized for the busiest hour (the same team every hour): the baseline;
- a fixed schedule sized a bit lower, at the 75% level ("fixed tuned");
- Erlang C + simple rules (morning and afternoon shifts + the overtime rule): the backup;
- Erlang C hour by hour: a theoretical reference.

Result: 40.1% cheaper than the fixed schedule, with the service target met (92.4% of chats answered within 20 seconds) and every labour rule respected. The main idea is kept.

Job 2: choosing the next customer (routing, goal DSO1)

When an advisor becomes free, who should they help first? CallVerse tests three *routers*:

Router	How it chooses	Role
FIFO with skill matching	"First in, first out": the oldest waiting customer first	Baseline
Skill score	A score that mixes the waiting time, the advisor's speed on that problem, the customer's personality, and live or overnight. Its weights are tuned by trying many random combinations in the twin.	Backup
MaskablePPO	An AI that learns by trial and error, only inside the simulator	Main idea

How the PPO router learns. Learning by trial and error, guided by a score, is called *reinforcement learning* (RL). Think of a video game:

In a video game	In CallVerse
What the player sees (the *state*)	The 10 oldest waiting customers, with 12 facts each (wait so far, problem family, personality, live or overnight, the advisor's speed on that family...), plus 7 facts about the centre (queue length, staff on duty, time of day...)
What the player does (the *action*)	Pick one of the 10 customers. Empty places are blocked (*masked*), so it can never make an impossible choice.
The score (the *reward*)	Minus points for every second a customer waits, and a big minus, worth 10 minutes of waiting, for each customer who hangs up
Practice (*training*)	300,000 decisions on busy single days (1.0 to 1.8 times the normal load, sudden surges, 5% to 30% of staff absent), from February 2017 to March 2018. Then a test on days it never saw: April to August 2018.

These game rules are called an *MDP* (Markov decision process). The twin can be played one step at a time (as a *Gymnasium environment*), so the same simulator is used to train the AI, to test it, and to show the live floor.

The result, honestly:

- On unseen stressed days, PPO cuts the average wait to answer (ASA) by 13.9% compared with FIFO.

- In full peak weeks (a backlog over several days, planned overtime), it does not reduce the mean wait (+2.2%), although it cuts the long waits (p90 wait 18% to 25% shorter).
- The switch rule asked for at least 10% less waiting in peak weeks. That is not met, so the skill-score router is the one for production. PPO stays a research track; the next step is to train it on several days in a row.

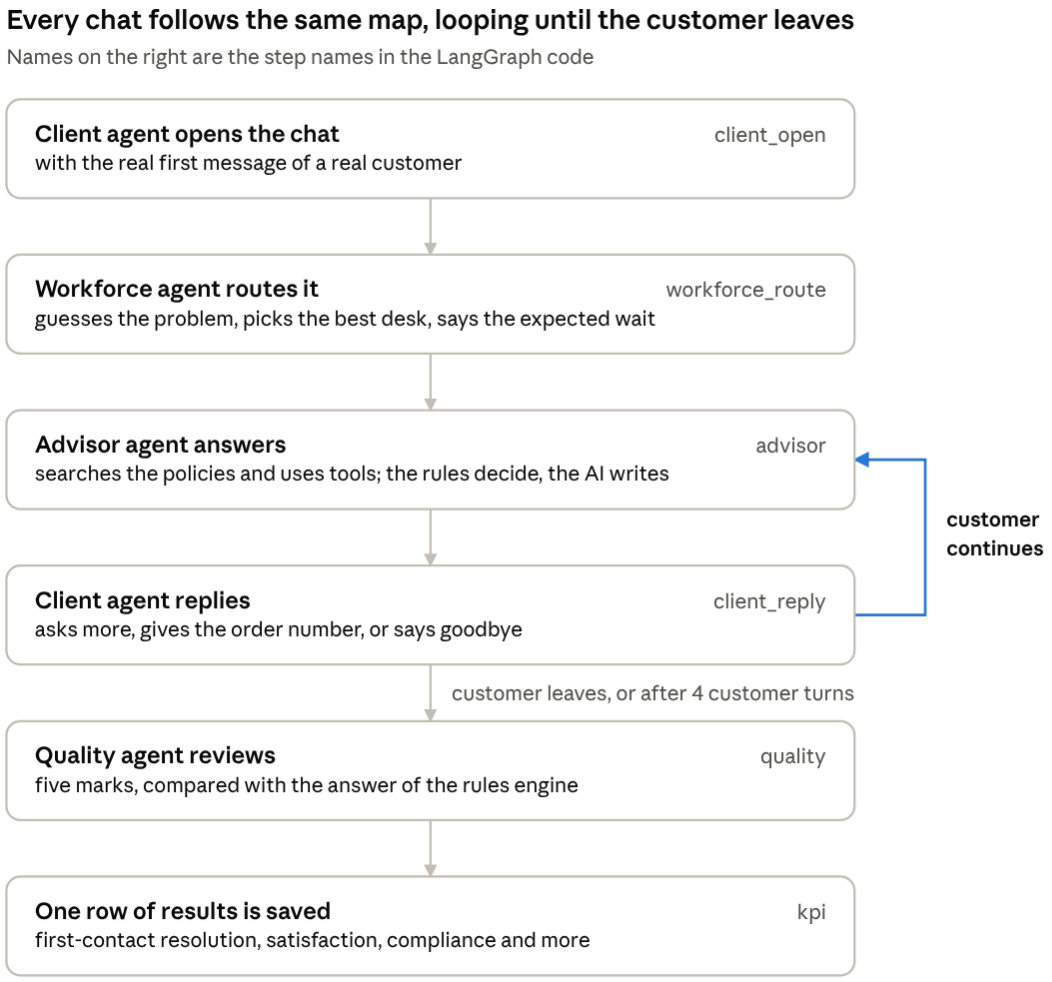
Original guide: parts 2.5, 2.6, 4.7 and 4.8.

9. The chat agents: a pretend customer, an advisor and a reviewer

Four AI agents take turns in every conversation, and the company's rules, not the AI, decide what the customer gets.

Think of a shop with a team of four:

- the **Client agent** is a *mystery shopper*: a pretend customer who tests the service;
- the **Workforce agent** is the *receptionist*: it sends the customer to the right desk and says how long the wait is;
- the **Advisor agent** is the *helper at the desk*: it answers the customer;
- the **Quality agent** is the *supervisor*: it reviews the conversation afterwards and gives marks.



one conversation · 6 steps, 1 loop

The advisor and the customer take turns until the customer leaves. Then the Quality agent grades the whole chat, and one row of results is saved.

The Client agent (the pretend customer). It knows a profile: the real first message, the order, the hidden problem, a personality and a tone. It answers the advisor with an AI model, in a fixed format (the message, "do I leave?", and a satisfaction score), or with simple rules when offline. Two safety rules protect it:

- When the advisor asks "your order number?", it always answers correctly, using rules, not the AI.
- If the AI starts talking like an advisor ("how can I help you?"), that turn is replaced by the rule-based customer.

At the end, it rates the chat from 1 to 5 (CSAT).

The Workforce agent (in a conversation). For each chat it guesses the problem family, picks the advisor desk with the best skill, and says the expected wait (Erlang C for that hour).

The Advisor agent, in five steps:

1. **Understand.** A small classifier (TF-IDF + logistic regression) guesses the intent. It is right 99.0% of the time on test sentences it never saw.
2. **Find.** It searches the policy library by meaning and takes the 3 best sections (part 10 explains how). If even the best match is weak (similarity below 0.57), it says the policies have no answer, instead of inventing one.
3. **Check.** It shares no order details before the customer gives the order number. This is rule 1 of the advisor playbook.
4. **Use tools.** It reads the database as it was at the contact time, with six tools: `lookup_order`, `get_ticket_history`, `check_refund_eligibility`, `open_ticket`, `search_knowledge_base`, and `recommend_resolution` (the company's rules engine).
5. **Reply** in plain, friendly language.

Three versions of the advisor:

Version	Who decides	Who writes	Role
template	The classifier and the rules	Approved, fixed answers	Backup: instant, always the same, works offline
rag	The AI model (which tools to use, what to say)	The AI model	The "pure" idea, kept for comparison
rag_policy	The rules engine	The AI model, which only puts the decision into words	Production (the hybrid)

Why "the rules engine decides, the AI writes"? In the first tests, the small AI alone invented order numbers and made promises the company does not allow. So in rag_policy, as soon as the order number is known, the program itself calls `lookup_order` and the rules engine. The AI receives the decision, for example "action: carrier investigation, hand over to: logistics, approved answer: ...", and only writes it nicely. It cannot change the decision.

Two more safety rules (*guardrails*) were added after the first runs:

- **Tool gating:** the order tools appear only after an order number is in the chat, because small models were calling them with invented numbers.
- **Verification guardrail:** a reply sent before the order is verified always ends by asking for the order number.

The Quality agent (the reviewer). It fills a scorecard with five marks from 1 to 5: compliance (rules followed, no invented promise), relevance, satisfaction, sentiment and resolution. It also says "correct?" and "resolved?", with one sentence of explanation. To

compare, it gets the right answer: what the rules engine says for the true intent. The AI judge (IBM Granite) comes from a different family than the advisor (SmolLM3), so it does not share the same blind spots. Offline, a rule-based checklist fills the same scorecard.

Can we trust the AI judge? CallVerse compares the judge's marks with reference marks, using two measures:

- **Cohen's quadratic kappa:** how much two graders agree, after removing the agreement you would get by luck. 1 = perfect, 0 = no better than chance. A 2-point disagreement counts 4 times more than a 1-point one.
- **Spearman's ρ** (rho): do the two graders put the conversations in the same order?

The rule: the judge is trusted only if kappa is 0.4 or more. Today it is below 0.4, so it is not trusted yet. Without human grades, the reference marks come from the generator's answer key (*proxy labels*). When 20 or more chats are graded by humans on the labeling page, the check switches to the human grades automatically.

Original guide: parts 2.7, 3.5, 3.6, 4.6 and 4.9.

10. The AI brains: search by meaning, and two small models on a laptop

CallVerse finds the right policy section by meaning, not by matching words, and runs two small AI models together on a 4 GB laptop graphics card.

Search by meaning, step by step:

1. A model called *bge-m3* turns any text into a list of 1,024 numbers, called an *embedding*. Texts with a similar meaning get similar lists, even when they use different words. bge-m3 works in many languages.
2. All 58 policy sections are turned into embeddings once, and stored in *FAISS*, a library for fast search among such lists.
3. When a customer writes, their message becomes an embedding too, and FAISS returns the closest sections.
4. "Closest" is measured with *cosine similarity*: 1 means the same direction (the same meaning), and a value near 0 means unrelated. The lists are scaled to length 1, so FAISS's quick "inner product" gives exactly the cosine.

An example from the original guide. "my parcel is late and nobody answers" shares almost no word with the section "Order late: estimated date has passed". Yet it is the right section. Keyword search (*TF-IDF*, the baseline) misses it; search by meaning finds it.

The result. On 255 test questions, search by meaning puts the right section in the top 4 results for 96.9% of the questions. Keyword search does it for 80.4%.

This search is the first half of *RAG* (retrieval-augmented generation): first find the right pages, then let the AI write the answer from them. It is like an open-book exam: the AI does not answer from memory.

The three models. *Parameters* are the numbers inside a model that it learned during training; more parameters usually means a stronger but heavier model.

Role	Model	Why this one
Advisor and pretend customer	SmolLM3-3B (Hugging Face), 3.1 billion parameters	Fully open, can use tools, strong for its size; one copy plays both roles
Judge	Granite-4.0-1B (IBM), 1.6 billion parameters	A different family from the advisor, so it does not prefer its own style; small enough to stay loaded
Search	bge-m3 (BAAI)	Embeddings in many languages; it runs on the processor (CPU) to leave the graphics card to the two others

Both chat models have the Apache-2.0 licence, which allows free use.

Fitting two models in a 4 GB graphics card (a GTX 1650):

- **4-bit quantization (NF4).** Normally each number of a model uses 16 bits of memory. NF4 stores it in 4 bits, with levels chosen to match the bell shape of the model's numbers. It is like saving a photo in a smaller file that still looks right. A 3-billion-parameter model drops from about 6 GB to about 2 GB. Both models together use about 3.2 GB of the 4 GB.
- **bfloat16 calculations.** With the usual *float16* format, numbers above 65,504 overflow (they are too big to store), and the models' output became "!!!!". *bfloat16* can hold numbers as big as the 32-bit format can, so the problem disappears, at the same speed on this graphics card.
- **Speed.** About 8 to 13 *tokens* (pieces of words) per second. One advisor reply takes about 7 to 10 seconds; a whole conversation with the four agents takes about 45 seconds.

The small details that make it work:

- **Chat templates.** Each model expects its own message format; the code builds it automatically. The "thinking" mode is turned off, for faster answers.

- **Tool calls.** The model writes a special tag to ask for a tool (`<tool_call>...</tool_call>`). The code reads it, runs the tool, and ignores unknown tools.
- **Streaming.** The reply appears on the screen piece by piece while it is being written. A filter hides the tool tags from the customer.
- **JSON answers.** *JSON* is a simple, structured text format. For the customer turns and the judge's scorecards, writing stops as soon as the JSON is complete. If a small model forgets to close it, a repair step closes it.
- **Cache.** Every AI call gets a fingerprint (a *SHA-256* hash of the model, its version, the messages, the tools, the temperature and the seed). An identical call is answered instantly from the cache, which also makes runs repeatable. The *temperature* sets how random the AI's writing is.
- **Tracing.** Every call is written to a log file (`llm_calls.jsonl`): which agent, which model, tokens, time, speed, tools, and whether the cache answered.
- **Pinning.** The command `callverse llm-pull` downloads the models and records their exact version. After that, the models load only from the local copy.
- **Safe fallback.** With no graphics card or no downloaded models, every agent switches to its offline backup, and the screen says so.

Original guide: parts 2.8 and 3.2.

11. Behind the scenes: how the parts talk to each other

A conversation follows a fixed map of steps, the browser and the server stay connected by an open line, and automatic checks test every change.

The conversation map (LangGraph). *LangGraph* runs a conversation as a map of steps: the diagram in part 9. The advisor ⇄ customer loop repeats until the customer leaves, or after 4 customer turns. At the end, one row of results is saved: first-contact resolution (*FCR*: solved in the first conversation), satisfaction (CSAT), compliance, policy-correct action, the judge's verdict, the tool success rate, the predicted intent, the number of turns and the expected wait. Running many conversations in a batch (`callverse agents`) gives the results of goal DSO2.

Live sessions: a human on either side. The same agents can also run one step at a time, so a human can play:

- the **customer**, in the front-office live chat;
- the **advisor**, at the advisor desk, with the AI as a *copilot* (it drafts each reply) or on *autopilot* (it answers alone).

The open line: WebSocket. The browser and the server talk through a *WebSocket* (`/ws/session`). Normal web requests are like letters: the browser asks, the server answers, and that is all. A WebSocket is like a phone call: the line stays open both ways. So the server can send each new piece of text, each agent step, the routing decision, the copilot's draft and the final review as soon as they happen. The browser sends: start, a customer message, an advisor message, "suggest a reply", autopilot, and end (with the satisfaction score).

Only one AI generation runs on the graphics card at a time: a global lock makes the others wait.

The API (FastAPI). *FastAPI* is the Python library used to build the server. The main groups of requests:

Requests	Used by	What they give
`/api/portal/...`, `/api/kb/...`, `/api/desk/queue`	Front office	Demo customers and their orders; help articles and search; the advisor's waiting queue
`/ws/session`	Front office and agent theater	Live conversations
`/api/overview`, `/api/data/...`, `/api/forecast`	Back office	Headline numbers, data charts, forecast
`/api/twin/...`	Back office	Run a scenario, compare options, replay one day
`/api/workforce/...`	Back office	Schedule and routing results
`/api/quality/...`, `/api/evaluation`, `/api/mlops`, `/api/system`	Back office	Labeling and judge checks, scorecard, experiments, models and graphics card

The full list, with a page to try each request, is at localhost:8000/docs (called *Swagger*).

The apps' code (the frontend):

- Three packages: `shared` (the common design), `front-office` and `back-office`. Both apps use React 18 + TypeScript + Vite. A change in `shared` appears in both apps at once.
- Colours are variables with a light set and a dark set, so the theme switches instantly. Chart colours are safe for colour-blind people, and each thing always keeps the same colour (for example, "Where is my order" is always blue).
- Animations use Framer Motion, and respect the computer's "reduce motion" setting.

- The live floor is drawn on a *canvas* at 60 images per second, because hundreds of moving dots would be too slow as normal page elements.
- Comfort features: a command palette (Ctrl+K), small pop-up messages, loading placeholders, and *error boundaries* (if one view breaks, the rest of the app keeps working). The fonts are stored locally, so the apps work offline on demo day.

Keeping the project reliable (MLOps and code quality):

- **MLflow** records every experiment: its settings, its results and its files (forecast backtests, PPO training, evaluations).
- **Repeatable results:** seeds in the settings files, exact model versions, the AI cache, and CP-SAT with one worker. The command `callverse all` rebuilds everything.
- **Docker Compose** packs everything into *containers* (api, web, mlflow + postgres) that run the same way on any computer. A second file, `docker-compose.gpu.yml`, adds the graphics card.
- **CI (GitHub Actions):** on every change pushed to GitHub, robots check the code style (ruff), the 5 layer rules (import-linter), 95+ tests (pytest, including a live WebSocket session) and the types, and they build both apps.

Original guide: parts 2.9, 2.11, 2.12 and 2.13.

12. Checking the work: how CallVerse proves its results

Every idea is compared with its baseline on the same pretend customers, over 10 runs, and a scorecard applies the switch rules automatically.

The four pieces of the evaluation:

1. **The KPI engine** computes averages, 95% confidence intervals and paired differences for every KPI.
2. **The experiments**: shift planners against the fixed schedule (DSO3); routers against FIFO, on stressed days and in peak weeks (DSO1); prepared against unprepared plans (BO4). For DSO2: search quality on 255 questions, conversations for each advisor version, the judge check, and how close the answers are to the policy answer.
3. **Component checks**: is each piece good on its own? Forecast error, search quality, judge agreement (kappa), twin realism (KS test, Erlang C error).
4. **The scorecard** applies each goal's switch rule and writes the verdicts in a file (`scorecard.json`), shown in both apps.

How search quality is scored:

- **recall@k**: the share of questions whose right section is in the first k results. Small example: recall@4 = 90% would mean that for 90 questions out of 100, the right section is among the first 4.
- **MRR** (mean reciprocal rank): if the right section comes 1st, the question scores 1; 2nd scores ½; 3rd scores ⅓, and so on. MRR is the average score. Higher is better.

The money of goal BO4 (in euros):

Saving = (labour cost of the unprepared plan − labour cost of the prepared plan) + €8 × the lost customers avoided

The *break-even value* of a lost customer is the value above which preparing pays off: €6 to €12, depending on the scenario. In the staff-shortage scenario the saving can be negative: preparing costs more extra labour than the €8 per customer brings back.

Where the numbers come from. The forecast, twin, schedule, routing and BO4 numbers come from the command `callverse evaluate` (10 seeds, 95% confidence intervals). The DSO2 conversation numbers were measured with the first AI models (qwen2.5 and llama3.2, through Ollama); the new run with the Hugging Face models is the next planned step. The search numbers do not depend on the AI model.

Honest limits: what is not perfect yet.

- The contacts, tickets and chats are created by the program, from real orders and real sentences, with written assumptions. Real logs could replace them without changing the pipeline.
- The models are small (3 billion and 1.6 billion parameters) and run on a laptop. So the hybrid design ("the rules decide, the AI writes") is necessary, and the AI judge is not trusted yet (kappa below 0.4). The next step is 60 conversations graded by humans.
- PPO is trained on single days. It needs training over several days in a row to beat FIFO in full peak weeks.
- The DSO2 conversation numbers in the report come from before the switch to the Hugging Face models; the new run is planned.

Original guide: parts 2.10, 4.10 and appendix D.

13. The screens, page by page

The front office is the shop side, for customers and advisors (light theme); the back office is the managers' control room (dark theme).

Front office (localhost:5173)

Page	What you see	Why it matters
Home	The help centre: a search box, the four problem families, 8 "try it as a real customer" cards (a real order and the real message its customer sent), and the advisor's 4 steps	The jury can "become" a real customer with one click
Semantic search	Results with their similarity score, the section title, and a short extract with the query words highlighted	Shows search by meaning in action
Help centre and articles	The policy documents, with a table of contents	Customers and the AI read the same rules
My orders	The demo customer's order as it was on the contact day: the journey placed → paid → given to the carrier → delivered, a late banner, previous contacts	The chat that follows matches what the advisor's tools will read
Live chat	A human customer chats with the AI advisor. A side panel shows each agent working, the routing decision, every step (search, tools, AI writing) and the quality review	The clearest demo of the four agents, and of "the rules decide, the AI writes"
Advisor desk	A queue of pretend customers. A human advisor answers them, with an AI copilot that drafts each reply, or an autopilot	Shows how much the human relied on the AI ("human · AI draft" or "human · edited AI draft")

Back office (localhost:5174)

Page	What you see	Why it matters
Overview	The 4 verdicts (2 of 4 main ideas kept), the data volumes, one card per goal, an animated map of the 5 layers	The one-screen summary for the jury

Page	What you see	Why it matters
Data and contracts	Weekly contacts per problem (peaks at Black Friday and Christmas 2017), contacts per hour, satisfaction scores, data checks	Where the contacts come from, and proof that the data is checked
Demand forecast	Real against forecast, error per model, error per distance (1 to 7 days ahead), next week's forecast, the clues that matter most	The forecast beats the simple method on data it never saw
Digital twin: live floor	One simulated day replayed: dots (customers) wait in a queue and fly to desks (advisors); a red ring means the customer waited more than 20 seconds	Makes the simulation visible; compares prepared and unprepared plans
Digital twin: scenario lab	What-if settings, KPI tiles with their 95% confidence intervals, a staffing heatmap by day and hour, "same customers, four decisions"	Tests ideas with statistics
Agent theater	The four agents handle a real contact with no human; the LangGraph map lights up step by step	This is how the DSO2 batches are evaluated
Workforce: shift planning	Cost, hours, the labour-rule checks, "required" and "scheduled" heatmaps, the roster	Shows where the 40% saving comes from
Workforce: routing	The game rules of PPO, wait and hang-up bars per router, the PPO learning curve	Compares the routers fairly
Quality and labeling	The judge agreement table (kappa), and a chat to grade by hand with 1 to 5 buttons	Decides if the AI judge can be trusted
Scorecard	The verdicts, the money saved by testing first, paired-difference tables, search quality, twin realism	All the results in one place

Page	What you see	Why it matters
Models and MLOps	Load the models on the graphics card, one card per model (version, size, memory), a GPU gauge, AI call traces, MLflow runs	Shows exactly which models run, and how fast

A *heatmap* is a grid of coloured squares: the darker the square, the bigger the number.

Original guide: parts 3 and 4, with a screenshot of every page.

14. Two full journeys, step by step

Following one customer message, and then one what-if test, from start to end shows how all the parts work together.

Journey 1: one customer message, from the keyboard to the quality review

1. **Browser.** The chat page opens a WebSocket (`/ws/session`) and sends "start" with the demo customer's contact. The customer is a human; the advisor is the AI.
2. **Server.** It builds the session: the customer's profile (order, date, true intent), the agents, and the tools set to the contact date.
3. **Browser.** The customer sends a message.
4. **Workforce agent** (first message only). It guesses the intent, then the problem family, then picks the best-skilled desk. It computes the expected wait with Erlang C.
5. **Advisor: find.** bge-m3 turns the customer's words into an embedding, and FAISS returns the 3 best policy sections.
6. **Advisor: check.** There is no order number yet, so only the policy search tool is allowed, and the AI is told to ask for the order number.
7. **AI model on the graphics card.** SmolLM3 writes the reply. The words stream to the screen, and the bubble types itself.
8. **Browser.** The customer gives the order number.
9. **Advisor: rules engine.** `lookup_order` reads the database as it was on the contact date. `recommend_resolution` applies the policy: for example, share the tracking, or open an investigation with the carrier if the parcel is late and not moving.
10. **AI model.** It receives the decision and the approved answer, and writes the reply in natural language. It cannot change the decision.
11. **Browser.** The customer clicks "End chat" and gives stars (the CSAT score).
12. **Quality agent.** Granite grades the conversation against the policy reference and fills the scorecard. The KPI row is computed, and the conversation is saved.

Journey 2: one what-if test, from a click to a confidence interval

1. In the scenario lab, the manager picks: promo spike, CP-SAT, FIFO, unprepared. Then they click Run.
2. The server builds the week's demand: the normal forecast for an unprepared plan, or the forecast adjusted to the scenario for a prepared plan.
3. Erlang C turns the hourly demand into the number of advisors needed each hour (80/20 target, occupancy at most 90%).
4. CP-SAT chooses the shifts and builds the roster under the labour rules. The plan becomes a duty table.
5. For each seed, the twin creates the scenario's customers (with the extra volume and the absences), and SimPy replays the week with the router, the overtime rule and each customer's patience.
6. The KPIs of each run become averages with 95% confidence intervals. To compare options, every option reuses the same seeds (common random numbers), and the paired differences decide what is significant.
7. The page shows the KPI tiles, the waiting and staffing charts, and the plan's heatmap. The live floor can replay any single day of that week.

Original guide: part 5.

15. Running it, and where the code lives

To see CallVerse working, you need four steps: download the AI models once, build everything with `callverse all`, start the server with `callverse serve`, then start the two apps.

The main commands, in order (the full setup is in `docs/RUN_GUIDE.md`):

1. `callverse llm-pull` downloads the AI models and records their exact version. You need it only once. Then `callverse status` checks the graphics card, the data and the models.
2. `callverse all` runs the whole pipeline: data → forecast → RL → evaluation (about 35 minutes). With `--skip-data` it keeps the data already built (about 20 minutes). With `--quick --skip-data` you get a demo-grade run in about 6 minutes.
3. `callverse serve` starts the server for both apps. Its documentation is at localhost:8000/docs.
4. In the `frontend` folder: `npm install` once, then `npm run dev:front` (front office, port 5173) and `npm run dev:back` (back office, port 5174), each in its own terminal.

Other useful commands:

Command	What it does
`callverse evaluate`	Runs every experiment, the component checks and the scorecard
`callverse agents --n 12 --variants rag_policy,rag`	Runs batches of AI conversations for goal DSO2
`callverse judge-calibrate --judge llm`	Measures how well the AI judge agrees with the labels
`docker compose up --build`	Runs everything in containers (add `-f docker-compose.gpu.yml` for the graphics card)

Watch out: only one program may hold the AI models on the 4 GB graphics card. Stop `callverse serve` before you run AI batches such as `callverse agents`.

Where the code lives:

Folder	What is inside
`configs/`	All the assumptions: data, simulation, agents (AI model settings), scenarios, app
`src/callverse/data/`	Notebook runner, data checks, loaders, database, ticket and chat generators
`src/callverse/forecasting/`	Clues (features), baselines, LightGBM, backtest, next-week forecast
`src/callverse/simulation/`	SimPy engine, routers, Erlang C, customers, twin setup
`src/callverse/agents/`	`client/`, `advisor/` (search, tools, classifier), `quality/`, `workforce/` (CP-SAT, PPO)
`src/callverse/llm/`	The Hugging Face models: 4-bit loading, chat templates, tool reading, streaming, cache, tracing
`src/callverse/orchestration/`	LangGraph conversation map, live sessions, shared runtime
`src/callverse/twin/`	Scenarios, scenario runner, live-floor replay, realism checks
`src/callverse/evaluation/`	KPI engine, experiments, component checks, scorecard

Folder	What is inside
`src/callverse/api/`	FastAPI routes and WebSockets
`frontend/`	`shared/` design, `front-office/`, `back-office/`, Dockerfile and nginx
`tests/`, `docs/`	Unit, smoke and integration tests; report, run guide, demo script, recruiter kit, guides

Original guide: appendices B and C.

Word list

Every hard word used in this guide, in alphabetical order, explained in one line.

Word	Plain meaning
Abandonment	A customer leaves the queue before an advisor answers.
AHT (average handle time)	How long one conversation takes, on average.
API	The list of requests a server accepts, like a restaurant menu.
ASA (average speed of answer)	The average wait of the customers who got an answer.
Backlog	The pile of messages left from the night, handled in the morning.
Backtest (rolling-origin)	Testing a forecast on the past many times, each time using only older data.
Baseline	The simple, normal method that a new idea must beat.
bfloat16	A 16-bit number format that can hold very big numbers without overflowing.
BO (business objective)	A business goal, such as "shorter waiting".
Break-even value	The value of a lost customer above which preparing pays off.
Cache	A store of past answers, so an identical question is answered instantly.

Word	Plain meaning
Chunk	One section of a policy document: the piece that search returns.
CI (continuous integration)	Automatic checks (style, rules, tests, builds) on every change.
Classifier	A model that puts a text into a category, such as its intent.
Common random numbers	Testing every option on exactly the same random customers.
Confidence interval (95%)	A range where the true value very likely lies.
Container (Docker)	A sealed box holding an app and everything it needs, so it runs the same anywhere.
Copilot / autopilot	The AI drafts replies for a human (copilot), or answers alone (autopilot).
Cosine similarity	How close two embeddings point in the same direction; 1 = same meaning.
CP-SAT	Google's solver that finds, and proves, the cheapest valid schedule.
CSAT	Customer satisfaction score, 1 to 5, from the survey after the chat.
Digital twin	A copy of a real system inside the computer, used to test decisions.
Discrete-event simulation	A simulation that jumps from one event to the next (SimPy).
DSO (data-science objective)	The data-science goal that serves a business goal, with a clear measure.
Embedding	A text turned into a list of numbers that captures its meaning.
Erlang	Unit of load: how many advisors would be busy all the time.
Erlang C	The formula that gives the chance that a customer must wait.
FAISS	A library for fast search among embeddings.

Word	Plain meaning
FastAPI	The Python library used to build the CallVerse server.
FCR (first-contact resolution)	The problem is solved in the first conversation, with no hand-over.
Feature	A clue that a model uses to make its prediction.
FIFO (first in, first out)	The oldest waiting customer is served first.
float16	The usual 16-bit number format; it overflows above 65,504.
Guardrail	A safety rule that stops the AI from doing something wrong.
Gymnasium environment	A standard "game" wrapper, so an RL agent can train on the simulator.
import-linter	A tool that checks that each code layer only uses the layers before it.
Intent	What the customer wants, for example "where is my order?".
JSON	A simple, structured text format that programs can read.
Kappa (Cohen's quadratic)	Agreement between two graders after removing luck; CallVerse trusts 0.4 or more.
KPI (key performance indicator)	A number that shows how well the business is doing.
KS test (Kolmogorov–Smirnov)	Checks if two lists of numbers follow the same pattern.
LangGraph	A library that runs agents as a map of steps.
Leakage	When a model uses information from the future by mistake.
LightGBM	A model made of many small decision trees, each one fixing earlier mistakes.
Little's law	Average number waiting = arrival rate × average wait.

Word	Plain meaning
LLM (large language model)	An AI that reads and writes text, like SmolLM3 or Granite.
LLM-as-judge	An AI model that grades another AI's answers with a scorecard.
Lognormal	A shape for durations: most are average, a few are very long.
MAE, RMSE, bias	Average error; error where big mistakes count more; too high or too low on average.
MaskablePPO	PPO (a trial-and-error learning method) that can never pick a forbidden choice.
MDP (Markov decision process)	The rules of the RL "game": state, action, reward.
MLflow	A tool that records each experiment's settings, results and files.
MLOps	The practices that keep models repeatable, tracked and easy to run.
MRR (mean reciprocal rank)	Search score: 1 if the right answer is 1st, ½ if 2nd, ⅓ if 3rd, averaged.
NF4 quantization	Storing a model's numbers in 4 bits instead of 16, to use less memory.
Occupancy	The share of working time that advisors spend in conversations.
p90 wait	90% of customers waited less than this.
Paired difference	The difference between two options, computed run by run on the same customers.
Pandera contract	A list of rules a data table must follow (types, ranges, unique ids).
Parameters	The numbers inside a model that it learned during training.
Persona	A pretend customer's personality: impatient, normal or patient.
Poisson	A pattern for random counts and arrivals: 0, 1, 2 and so on per period.

Word	Plain meaning
Policy-correct action	The advisor's outcome is the same as the rules engine's outcome.
Proxy labels	Reference marks taken from the generator's answer key, used before humans grade.
RAG (retrieval-augmented generation)	Find the right documents first, then let the AI answer from them.
recall@k	The share of questions whose right answer is in the first k results.
Reinforcement learning (RL)	Learning by trial and error, guided by a score (the reward).
Roster	The list of which named advisor works which shift.
Router	The rule that picks which waiting customer a free advisor takes.
Rules engine	A program with the company's rules that decides the correct action.
Scenario	A "what if" version of a real week (promo spike, delays, absences…).
Seed	The starting number of the random generator: same seed, same results.
Service level 80/20	The share of chats answered within 20 seconds; the target is 80%.
Significant	A difference whose 95% confidence interval does not contain 0: real, not luck.
Solver	A program that searches many combinations and finds the best one.
Spearman's ρ	Do two graders put the items in the same order?
SQLite	A small database stored in one file.
Streaming	Showing the AI's reply piece by piece, while it is written.
Swagger	The web page that lists and tests every request of the API.

Word	Plain meaning
Switch rule	A test fixed in advance that decides which method is kept.
Synthetic data	Data created by a program (here, from real orders and real sentences).
Temperature	How random the AI's writing is.
TF-IDF	A classic way to turn text into numbers by counting words, with more weight for rare words.
Token	A small piece of a word; AI models read and write tokens.
Tool call	The AI asks the program to run a function, like looking up an order.
WebSocket	A connection that stays open both ways, like a phone call.
wMAPE	Total forecast error divided by total real contacts.
YAML	A simple text format for settings files.

Original guide: appendix A.